

State Space Search

1. Overview

Here, **we'll discuss the state space search concept in detail with an example.**

Furthermore, we'll present several applications where we can use this concept.

2. State Space Search

A state space is a mathematical representation of a problem that defines all possible states that the problem can be in. Furthermore, in search algorithms, we use a state space to represent the current state of the problem, the initial state, and the goal state. Additionally, we represent each state in the state space by a set of variables.

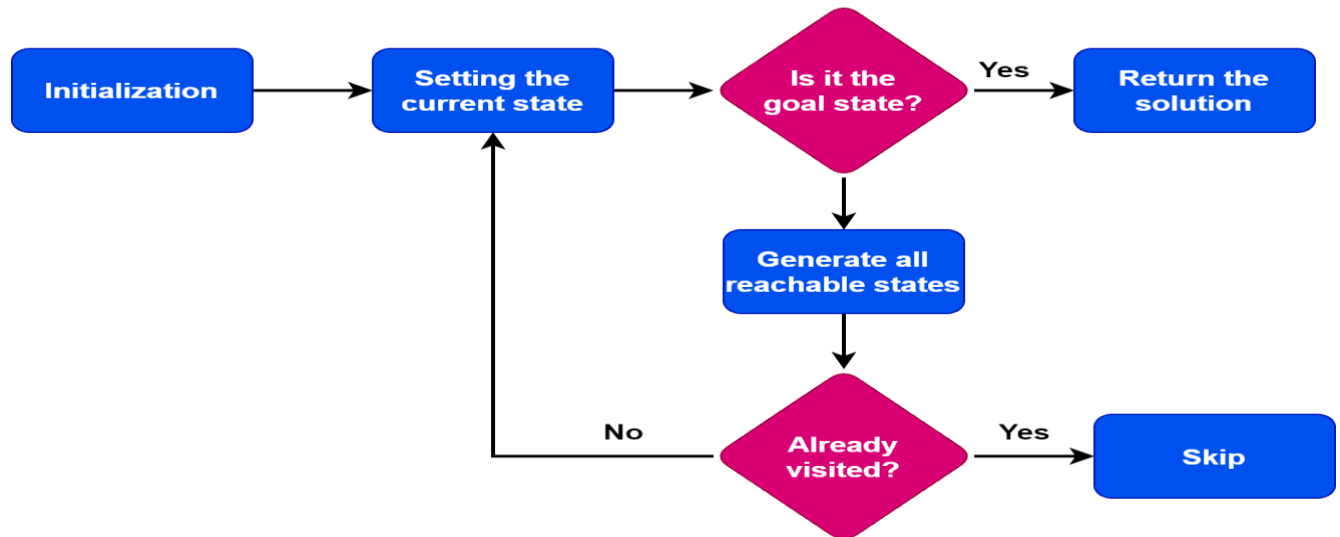
State space search is a method used widely in [artificial intelligence](#) and [computer science](#) to find a solution to a problem by searching through the set of possible states of the problem. Furthermore, a state space search algorithm uses the state space to navigate from the initial state to the goal state. Additionally, it generates and explores possible successors of the current state until we find a solution.

The state space size can greatly affect a search algorithm's efficiency. Hence, it's important to choose an appropriate representation and search strategy to efficiently search the state space.

The most famous state space search algorithm is the [A* algorithm](#). Other popular state space search algorithms are [breadth-first search \(BFS\)](#), [depth-first search \(DFS\)](#), [hill climbing](#), [simulated annealing](#), and [genetic algorithms](#).

3. Steps

Now let's discuss the steps of a typical state space search algorithm:



The first step is to initialize the search by setting the initial state as the current state. After the initialization step, we check if the current state is a goal state. Finally, if the current state is a goal state, we terminate the algorithm and return the result.

However, if the current state is not the goal state, we generate the set of possible states that can be reached from the current state. Additionally, these states are known as successor states. Furthermore, for each successor state, we check if it has been previously visited. If the state is already explored, we skip the state. If it has not been visited, we add it to the queue of states to be visited.

Moving forward, we set the next state in the queue as the current state and check if it's a goal state. If we find the goal state, we return the result. Otherwise, we repeat the previous step until we find the goal state or finish exploring all the states. Furthermore, if all possible states have been explored and we can't reach the target state, we return with no solution.

The specific implementation details of the algorithm depend on the problem. Additionally, **the algorithm's performance depends on the data structures we use to represent the states and keep track of the search.**

4. Example

A common example of a state space search is the 8-puzzle problem. The 8-puzzle is a sliding puzzle that consists of 8 numbered tiles in a 3, 3 grid and one blank space. The goal is to rearrange the tiles from a given initial state to a final goal state by sliding the tiles into the blank space.

In this problem, we represent the state space by the 9 tiles in the puzzle and their positions in the grid. Additionally, we present each state in the state space by a 3 3 array with values from 1 to 8. Finally, we represent a blank space with an empty tile.

The initial state represents the starting configuration of the tiles. The goal state represents the desired configuration. Furthermore, the search algorithms use the state space to find a sequence of moves that transform the initial state into the goal state.

For example, we can use the breadth-first search to explore all possible states reachable from the initial state. It explores all the states one by one in a sequence. It ensures the solution but can become very slow for large state spaces. Other algorithms, such as A* search, use [heuristics](#) to guide the search more efficiently.

We're taking a practical example of an 8-puzzle problem. Let's pick current and target states for this example:

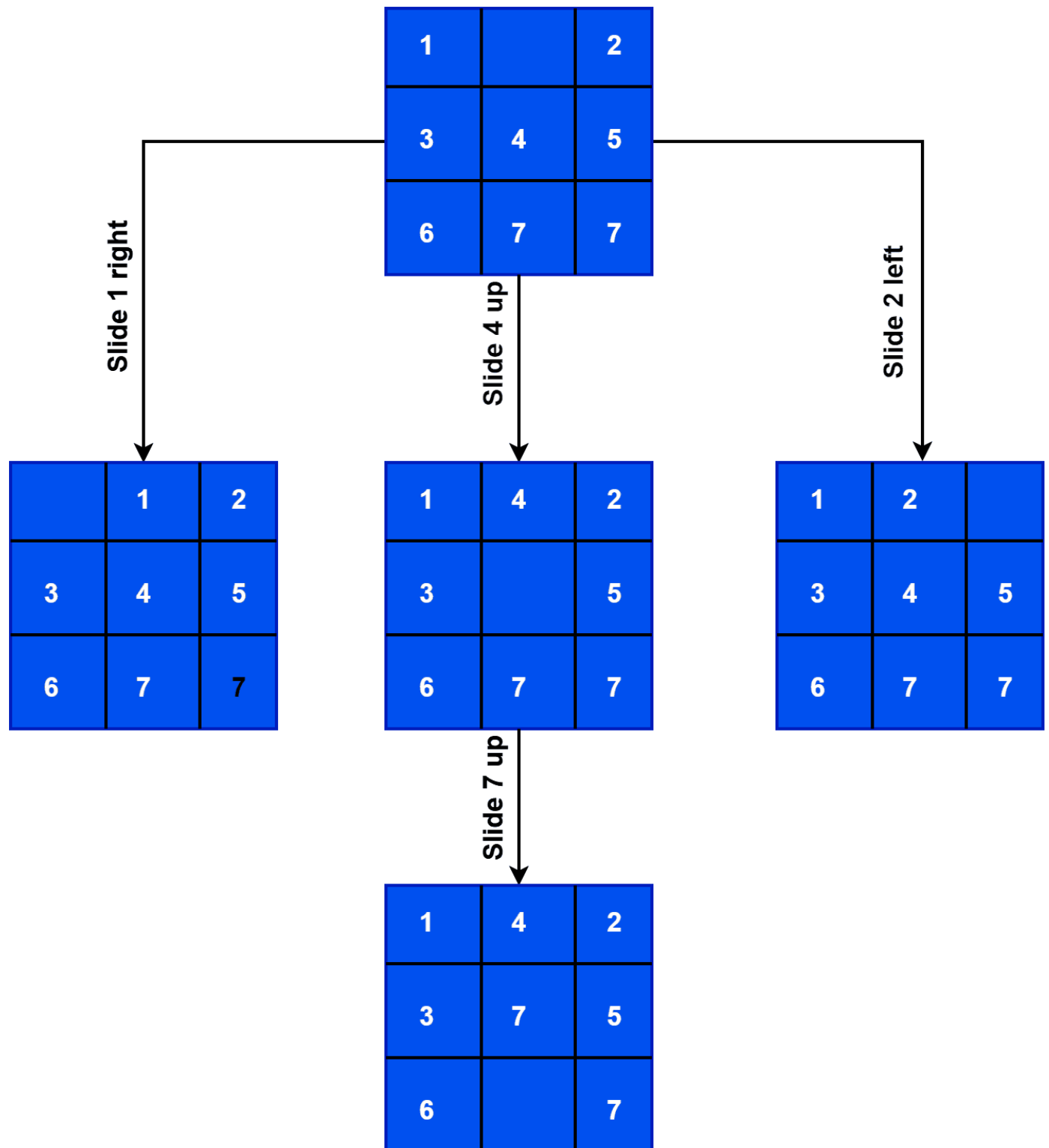
1		2
3	4	5
6	7	7

Current State

1	4	2
3	7	5
6		7

Target State

Here, we aim to start from the current state and reach the target state by sliding the numbers through the blank. Furthermore, the approach is to explore all the states we can reach from the current state. For each new state, we check if it's the target state or not. Let's take a look at how to reach the target state from the current state:



5. Applications

State space search algorithms have a wide range of applications in various fields, including artificial intelligence, robotics, game playing, computer networks, operations research, bioinformatics, cryptography, and supply chain management.

In artificial intelligence, we can use state space search algorithms to solve problems such as [pathfinding](#), [planning](#), and [scheduling](#). Additionally, we can employ space search algorithms to plan the motion of robots, determining the best sequence of actions to achieve a goal. Furthermore, we also use these algorithms in games to determine the best move for a player, given a particular game state.

In computer networks, we can take advantage of state space search algorithms to optimize [routing](#) and [resource allocation](#) in computer networks. Additionally, we also use them in operations research to solve [optimization problems](#), such as scheduling and resource allocation. We can also apply these algorithms to find patterns in biological data and predict [protein structures](#) in bioinformatics.

In order to find solutions to [cryptographic problems](#), such as breaking codes and finding cryptographic keys, we can employ these algorithms. Furthermore, we can use these algorithms to optimize the flow of goods and resources in logistics and supply chain management.

These are just a few examples of the many applications of state space search algorithms. The ability to efficiently search large state spaces and find solutions to complex problems makes state space search algorithms a powerful tool in a wide range of fields.

6. Conclusion

In this tutorial, we discussed the state space search concept in detail with an example. Furthermore, we explored different fields and applications where it can be extremely useful.

Depth-First Search (DFS) and Breadth-First Search (BFS)

1. Introduction

Depth-First Search (DFS) and Breadth-First Search (BFS). Here, we'll compare them and discuss in which scenarios we should use one instead of the other.

2. Search

Search problems are those in which our task is to find the optimal path between a start node and a goal node in a graph. There are several variations to search problems: the graph may be directed or undirected, weighted or unweighted, and there may be more than one goal node.

DFS and BFS are suitable for unweighted graphs, so we use them to find the shortest path between the start and the goal.

3. Depth-First Search and Breadth-First Search

Both algorithms search by superimposing a tree over the graph, which we call the search tree. DFS and BFS set its root to the start node and grow it by adding the successors of the tree's current leaves. In that way, DFS and BFS cover the whole graph until they find the goal node or exhaust the graph. What they differ in is the order in which they visit the nodes, i.e., add them to the search tree.

In each step, **DFS adds to the tree a child of the most recently included node with at least one unincluded child.** So, DFS adds the start node, then its child, then its grandchild, and so on. For that reason, DFS increases the depth of the search tree in each step as much as it can. Then, if there aren't more children of a node to add, it backtracks to the node's parent.

In contrast, BFS grows the tree layer by layer. It first adds all the start node's children, thus completing level 1. Then, it adds all the children of all the leaves at the first level one by one. Afterward, it adds all the children of the start node's grandchildren, and so on. So, if the branching factor is constant across all the levels, BFS makes the tree wider at each step. To do so, BFS does the opposite of DFS: **it adds to the tree a child of the least recently added node that has at least one unincluded child.**

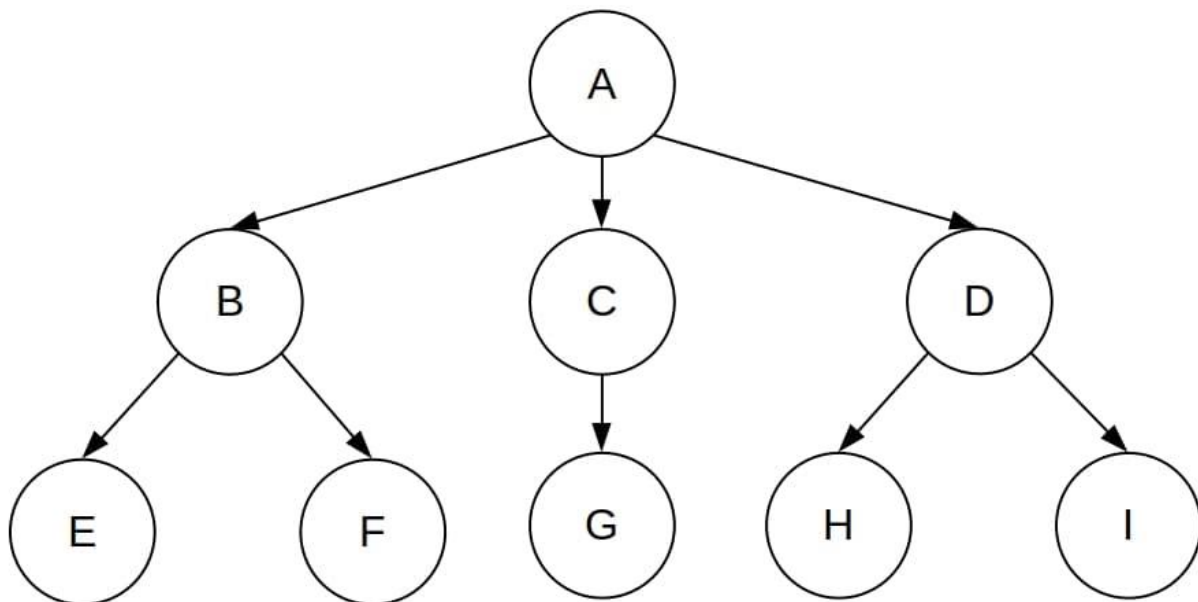
3.1. Tree-Like Search vs. Graph Search

Both algorithms may come in two variants: [Tree-Like Search](#) and [Graph Search](#). The Tree-Like Search (TLS) versions don't check for repeated nodes and may include the same node more than once in the tree. That is why they're prone to get stuck in a loop. In contrast, the Graph-Search (GS) alternatives check for repetitions, so they're immune to cycles but require more memory.

We'll focus on the TLS versions since it's easier to spot the differences between them.

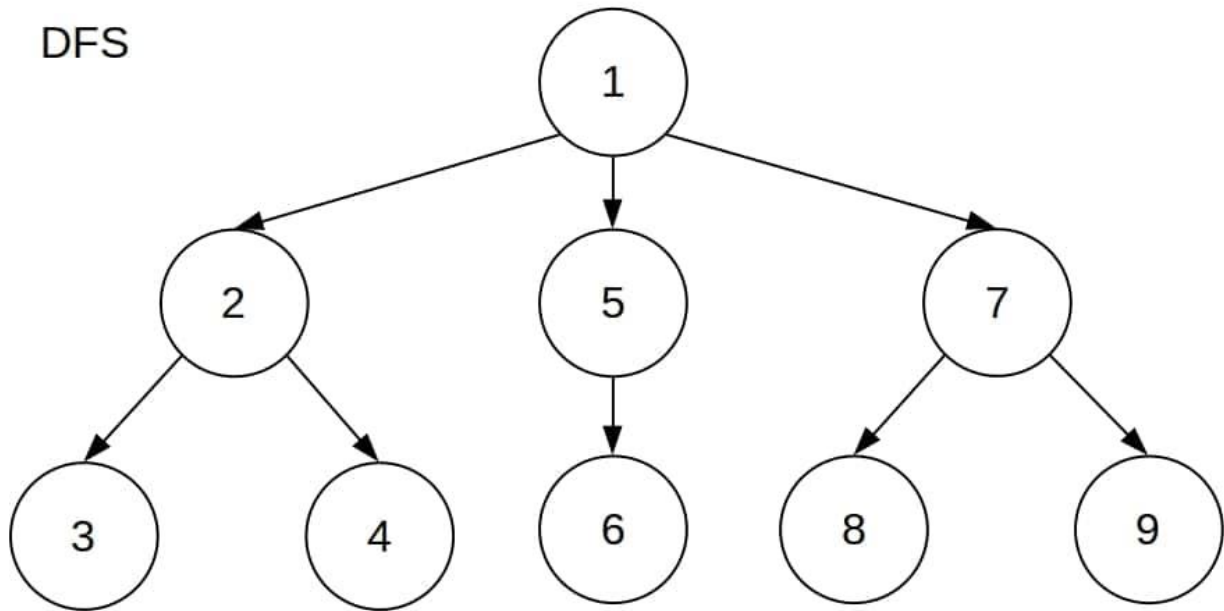
4. Conceptual Difference

The following example illustrates the main conceptual difference between DFS and BFS. Let's imagine that we've got the following search tree:

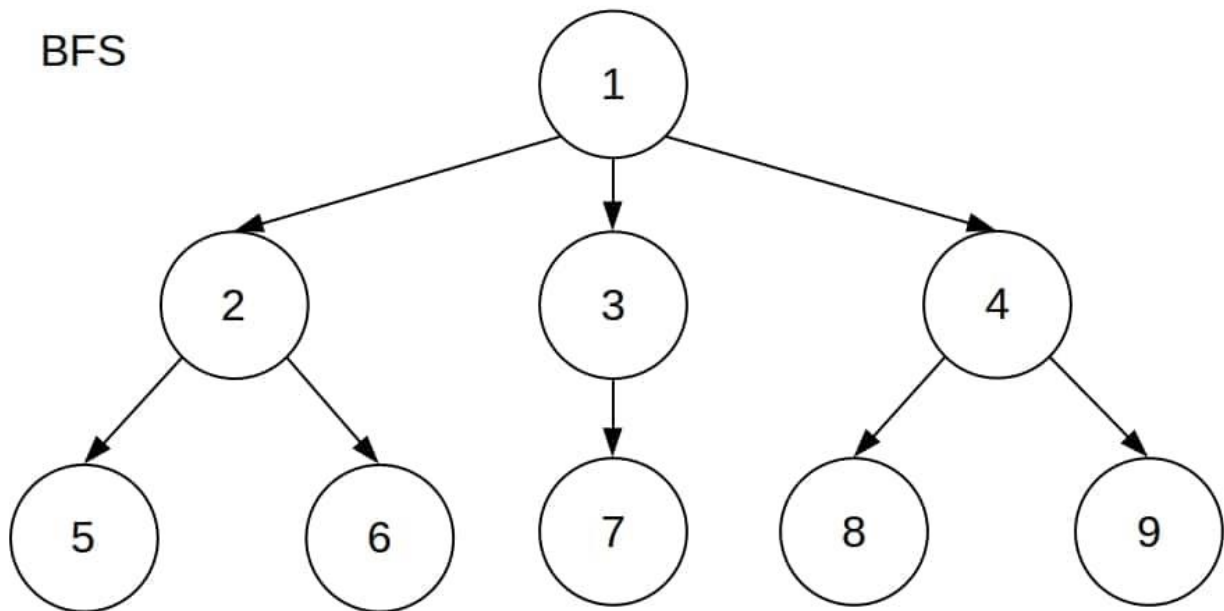


Let's mark each node by the ordinal number of its inclusion in the tree. The orders in which DFS and BFS include the nodes differ:

DFS



BFS



In the BFS tree, all the inclusion numbers at level 1 are lower than the numbers at level 2. That's not the case in DFS. In DFS, if node 1 has a lower inclusion number than node 2, then all the descendants of 1 have lower numbers than 2 and its descendants.

So, BFS grows the tree level by level, whereas DFS grows it sub-tree by sub-tree.

5. Implementation Differences

At each execution step, both DFS and BFS maintain what we call the search frontier. **It's a set of nodes we identified as children of the nodes currently in the tree but didn't add to it yet.** Since BFS always adds the child of the least recently included node, it uses a FIFO queue to maintain the frontier. In contrast, DFS uses a LIFO queue in its iterative implementation.

When adding a node to the search tree, we also identify its children and add them to the frontier. That's called expanding the node. So, we can describe BFS and DFS as follows:

- DFS expands the deepest node in the frontier.
- BFS expands the shallowest node.

DFS is also very suitable for a recursive implementation. In that case, the call stack acts as a frontier.

6. Differences in Completeness and Optimality

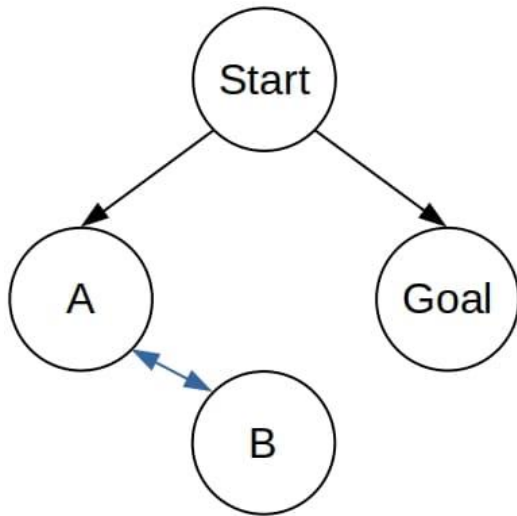
We say that a search algorithm is complete if it always terminates. That means that the algorithm finds a path to the goal if at least one goal is reachable from the start. Otherwise, it returns a notification of failure.

If an algorithm returns the optimal path to a goal node, provided that at least one is present in the graph, we say that the algorithm is optimal.

6.1. Completeness

DFS isn't a complete algorithm. To see why let's imagine that the graph contains a cycle. Depending on the choice of the successor to add first to the tree when considering the successors of a node, DFS may get stuck in a loop:

Even if we randomize the choice, we can still end in a loop and run indefinitely, although such an outcome is less likely. So, DFS may never terminate even if the goal is very close to the start node and the graph is finite and small:



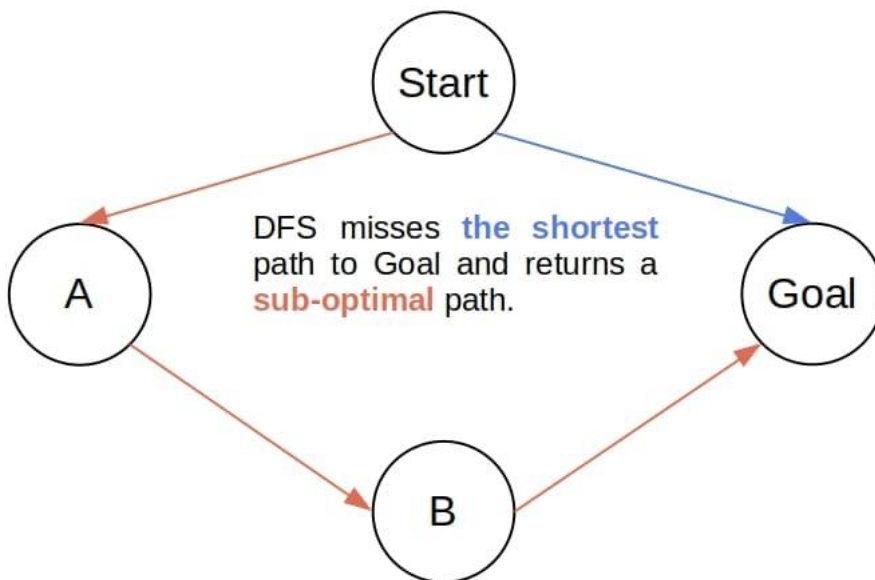
If (the TLS version of) DFS goes to `B`, it will forever alternate between `A` and `B`.

On the other hand, **BFS is always complete if the graph is finite**. If the graph is infinite, then BFS is complete under the following condition:

A goal node is reachable from the start, and no node has an infinite number of successors.

6.2. Optimality

DFS isn't an optimal algorithm either. It may return a sub-optimal path to the goal, and that happens if the goal is reachable in more than one way, but DFS discovers a longer path first:



In contrast, **BFS is optimal**.

7. Complexity Differences

Before we analyze **complexity**, let's introduce the notation. Let b be the tree's branching factor, l the length of the longest path in the graph, and d the depth of the shallowest goal node. If no goal is reachable from the start, we'll consider that $d = l$. Also, we'll consider only the cases where the algorithms terminate.

7.1. Time Complexity

In the worst-case scenario, DFS creates a search tree whose depth is d , so its time complexity is $O(b^d)$.

Since BFS is optimal, its worst-case outcome is to find the goal node after adding b nodes at level 1, b^2 at level 2, and so on up to level d where it adds b^d nodes. In total, the worst-case tree of BFS contains $\sum_{i=0}^d b^i$ nodes, which means that the time to grow it is of the order $O(b^{d+1})$.

7.2. Space Complexity

Since BFS keeps adding more nodes to the frontier but pops the oldest among them, it has to store all of them. So, **its space complexity in the worst-case scenario is $O(b^d)$** because that's the worst-case size of the frontier.

DFS is different. After exploring the whole subtree whose root is node n , DFS tries the siblings of n . But, while processing n and its descendants, DFS doesn't have to keep the descendants of the n 's siblings in the main memory. It can focus only on the currently active path, and all it needs for backtracking are the siblings of the nodes on it. **So, the worst-case space complexity of DFS is $O(b \cdot d)$.**

We can reduce the complexity of DFS if we yield the children of the deepest node on the currently active path one by one instead of returning them all at once. In that case, we store no more than b nodes at any point during execution. And we can go even further! If we can transform the nodes to one another, we can use a single node and apply transformations when replacing it with its child or siblings. As a result, we get an $O(b)$ space complexity, but the transformations have to be reversible because of backtracking.

8. How to Choose Between DFS and BFS?

All this said, the actual question we're interested in is how to choose between DFS and BFS.

If we know that the goal is deep, we should use DFS because it reaches deep nodes faster than BFS. A good example is constraint satisfaction. In those problems, we have to assign values to several variables such that the constraints on the variables and their values are satisfied. Each node in the resulting search graph is a list of assignments. Since we're interested in complete solutions, it makes more sense to reach the full assignment nodes as fast as possible with DFS than to slowly search through incomplete assignments with BFS.

On the other hand, if we know that the goal may appear at a shallow level in the search tree, BFS is a better choice. The same goes if we cannot accept a sub-optimal path and are ready to spend more memory to get the theoretical guarantees. But, the caveat is that BFS can require a lot of memory and will prove inefficient if the branching factor is too large or the goal is deep.

It's precisely the memory complexity that makes BFS impractical even though it's complete and optimal in the cases where DFS isn't. For that reason, DFS has become the working horse of AI. We can even sort out the problems of its incompleteness and sub-optimal solutions. [Iterative deepening](#), which limits the depth of DFS and runs it with incremental limiting depths, is complete, optimal, and of the same time complexity as BFS. However, its memory complexity is the same as that of DFS.

8. Conclusion

In this article, we compared Depth-First Search (DFS) to Breadth-First Search (BFS). While BFS has some theoretical advantages over DFS, it's impractical because of the high order of its space complexity. That is why we use DFS more often.